

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
"Белгородский государственный технологический университет им. В. Г.
Шухова"
(БГТУ им. В.Г. Шухова)



Институт энергетики, информационных технологий и управляющих систем

Кафедра программного обеспечения вычислительной техники
и автоматизированных систем

Лабораторная работа № 5
по дисциплине: Исследование операций
по теме: Двойственный симплекс метод

Выполнил: студент группы ПВ-221
Дорошенко Владимир Юрьевич
Проверил: ст. преподаватель:
Вирченко Ю.В.

Белгород 2024

Цель работы: изучить элементы теории двойственности, двойственный симплекс-метод для пары симметрично двойственных задач, а также метод последовательного уточнения оценок.

Задания для подготовки к работе

1. Изучить правило составления двойственных задач, а также формулировки и применения первой, второй и третьей теорем двойственности.
2. Изучить двойственный симплекс-метод для симметрично двойственных задач. Составить и отладить программу решения пары симметрично двойственных задач двойственным симплекс-методом.
3. Изучить понятие псевдоплана, построение симплекс-таблицы, отвечающей псевдоплану. Освоить метод последовательного уточнения оценок. Составить и отладить программу решения задачи ЛП методом последовательного уточнения оценок.
4. Для подготовки тестовых данных решить вручную одну из следующих ниже задач двойственным симплекс-методом для пары симметрично двойственных задач, а также методом последовательного уточнения оценок.

7.

$$z = 2x_1 - 7x_2 - x_3 - 4x_4 \rightarrow \max;$$

$$\begin{cases} -3x_1 + x_2 - 4x_3 + x_4 = 18, \\ 2x_1 + 3x_2 + 4x_3 + 2x_4 \geq 32, \\ 4x_1 + 5x_2 + 2x_3 - 3x_4 = 24, \\ x_i \geq 0 \ (i = \overline{1, 4}). \end{cases}$$

Лабораторная работа №5

Вариант 7

$$Z = 2x_1 - 7x_2 - x_3 - 4x_4 \rightarrow \max$$

$$\begin{cases} -3x_1 + x_2 - 4x_3 + x_4 = 18 \\ 2x_1 + 3x_2 + 4x_3 + 2x_4 \geq 32 \quad (\cdot -1) \\ 4x_1 + 5x_2 + 2x_3 - 3x_4 = 24 \end{cases}$$

$$\begin{cases} 2x_1 + 3x_2 + 4x_3 + 2x_4 \geq 32 \quad (\cdot -1) \\ 4x_1 + 5x_2 + 2x_3 - 3x_4 = 24 \end{cases}$$

$$x_i \geq 0 \quad (i = \overline{1, 4})$$

$$\begin{cases} -3x_1 + x_2 - 4x_3 + x_4 = 18 \mapsto y_1 \\ -2x_1 - 3x_2 - 4x_3 - 2x_4 \leq -32 \mapsto y_2 \\ 4x_1 + 5x_2 + 2x_3 - 3x_4 = 24 \mapsto y_3 \end{cases}$$

$$Z = 2x_1 - 7x_2 - x_3 - 4x_4 \rightarrow \max$$

$$\begin{cases} y_1 - 2y_2 - 3y_3 \geq -4 \\ -3y_1 - 2y_2 + 4y_3 \geq 2 \\ y_1 - 3y_2 + 5y_3 \geq -7 \\ -4y_1 - 4y_2 + 2y_3 \geq -1 \\ y_j \geq 0 \quad (j = \overline{1, 3}) \end{cases}$$

$$\widetilde{L}(\overline{Y}) = 18y_1 - 32y_2 + 24y_3 \rightarrow \min$$

$$\begin{cases} 3y_1 + 2y_2 - 4y_3 + y_4 = -2 \\ -y_1 + 3y_2 - 5y_3 + y_5 = 7 \\ 4y_1 + 4y_2 - 2y_3 + y_6 = 1 \\ -y_1 + 2y_2 + 3y_3 + y_7 = 4 \end{cases}$$

Матрица коэффициентов $A = a_{ij}$ этой системы уравнений имеет вид:

3	2	-4	1	0	0	0
-1	3	-5	0	1	0	0
4	4	-2	0	0	1	0
-1	2	3	0	0	0	1

Решим систему уравнений относительно базисных переменных: x_4, x_5, x_6, x_7

Полагая, что **свободные переменные** равны 0, получим первый опорный план:

$X_0 = (0, 0, 0, -2, 7, 1, 4)$

Базисное решение называется допустимым, если оно неотрицательно.

Базис	B	y_1	y_2	y_3	y_4	y_5	y_6	y_7
y_4	-2	3	2	-4	1	0	0	0
y_5	7	-1	3	-5	0	1	0	0
y_6	1	4	4	-2	0	0	1	0
y_7	4	-1	2	3	0	0	0	1
$F(Y_0)$	0	-18	32	-24	0	0	0	0

1. Проверка критерия оптимальности.

План 0 в симплексной таблице **является псевдопланом**, поэтому определяем ведущие строку и столбец.

2. Определение новой свободной переменной.

Среди отрицательных значений базисных переменных выбираем наибольший по модулю. Ведущей будет 1-ая строка, а переменную x_4 следует вывести из базиса.

3. Определение новой базисной переменной.

Минимальное значение θ соответствует 3-му столбцу, т.е. переменную x_3 необходимо ввести в базис.

На пересечении ведущих строки и столбца находится разрешающий элемент (РЭ), равный (-4).

Базис	B	y_1	y_2	y_3	y_4	y_5	y_6	y_7
y_4	-2	3	2	-4	1	0	0	0
y_5	7	-1	3	-5	0	1	0	0
y_6	1	4	4	-2	0	0	1	0
y_7	4	-1	2	3	0	0	0	1
F(Y0)	0	-18	32	-24	0	0	0	0
θ		-	-	$-24 : (-4) = 6$	-	-	-	-

4. Пересчет симплекс-таблицы.

Выполняем преобразования симплексной таблицы методом Жордано-Гаусса.

Базис	B	y_1	y_2	y_3	y_4	y_5	y_6	y_7
y_3	1/2	-3/4	-1/2	1	-1/4	0	0	0
y_5	19/2	-19/4	1/2	0	-5/4	1	0	0
y_6	2	5/2	3	0	-1/2	0	1	0
y_7	5/2	5/4	7/2	0	3/4	0	0	1
F(Y0)	12	-36	20	0	-6	0	0	0

Представим расчет каждого элемента в виде таблицы:

В	x_1	x_2	x_3	x_4	x_5	x_6	x_7
$-2 : -4$	$3 : -4$	$2 : -4$	$-4 : -4$	$1 : -4$	$0 : -4$	$0 : -4$	$0 : -4$
$7 - (-2 * -5) : -4$	$-1 - (3 * -5) : -4$	$3 - (2 * -5) : -4$	$-5 - (-4 * -5) : -4$	$0 - (1 * -5) : -4$	$1 - (0 * -5) : -4$	$0 - (0 * -5) : -4$	$0 - (0 * -5) : -4$
$1 - (-2 * -2) : -4$	$4 - (3 * -2) : -4$	$4 - (2 * -2) : -4$	$-2 - (-4 * -2) : -4$	$0 - (1 * -2) : -4$	$0 - (0 * -2) : -4$	$1 - (0 * -2) : -4$	$0 - (0 * -2) : -4$
$4 - (-2 * 3) : -4$	$-1 - (3 * 3) : -4$	$2 - (2 * 3) : -4$	$3 - (-4 * 3) : -4$	$0 - (1 * 3) : -4$	$0 - (0 * 3) : -4$	$0 - (0 * 3) : -4$	$1 - (0 * 3) : -4$
$0 - (-2 * -24) : -4$	$-18 - (3 * -24) : -4$	$32 - (2 * -24) : -4$	$-24 - (-4 * -24) : -4$	$0 - (1 * -24) : -4$	$0 - (0 * -24) : -4$	$0 - (0 * -24) : -4$	$0 - (0 * -24) : -4$

1. Проверка критерия оптимальности.

Текущий опорный план неоптимален, так как в индексной строке находятся положительные коэффициенты.

2. Определение новой базисной переменной.

В качестве ведущего выберем столбец, соответствующий переменной x_2 , так как это наибольший коэффициент.

3. Определение новой свободной переменной.

3-ая строка является ведущей.

Разрешающий элемент равен (3) и находится на пересечении ведущего столбца и ведущей строки.

Базис	В	y_1	y_2	y_3	y_4	y_5	y_6	y_7	min
y_3	$1/2$	$-3/4$	$-1/2$	1	$-1/4$	0	0	0	-
y_5	$19/2$	$-19/4$	$1/2$	0	$-5/4$	1	0	0	19
y_6	2	$5/2$	3	0	$-1/2$	0	1	0	$2/3$
y_7	$5/2$	$5/4$	$7/2$	0	$3/4$	0	0	1	$5/7$
F(x_1)	12	-36	20	0	-6	0	0	0	

4. Пересчет симплекс-таблицы.

Получаем новую симплекс-таблицу:

Базис	В	y_1	y_2	y_3	y_4	y_5	y_6	y_7
y_3	5/6	-1/3	0	1	-1/3	0	1/6	0
y_5	55/6	-31/6	0	0	-7/6	1	-1/6	0
y_2	2/3	5/6	1	0	-1/6	0	1/3	0
y_7	1/6	-5/3	0	0	4/3	0	-7/6	1
F(Y1)	-4/3	-158/3	0	0	-8/3	0	-20/3	0

1. Проверка критерия оптимальности.

Среди значений индексной строки нет положительных. Поэтому эта таблица определяет оптимальный план задачи.

Окончательный вариант симплекс-таблицы:

Базис	В	y_1	y_2	y_3	y_4	y_5	y_6	y_7
y_3	5/6	-1/3	0	1	-1/3	0	1/6	0
y_5	55/6	-31/6	0	0	-7/6	1	-1/6	0
y_2	2/3	5/6	1	0	-1/6	0	1/3	0
y_7	1/6	-5/3	0	0	4/3	0	-7/6	1
F(Y2)	-4/3	-158/3	0	0	-8/3	0	-20/3	0

Оптимальный план можно записать так:

$$y_1 = 0, y_2 = 2/3, y_3 = 5/6$$

$$F(Y) = 18 \cdot 0 - 32 \cdot 2/3 + 24 \cdot 5/6 = -4/3$$

Теперь из двойственной задачи находим ответ для изначальной задачи

$$-4/3 + 20/6 \geq 2 - x_1 = 0$$

$$-8/3 + 10/6 \geq -1 + x_2 \neq 0$$

$$-2 + 25/6 \geq -7 - x_3 = 0$$

$$-4/3 - 15/6 \geq -4 + x_4 \neq 0$$

Подставляем в исходное уравнение

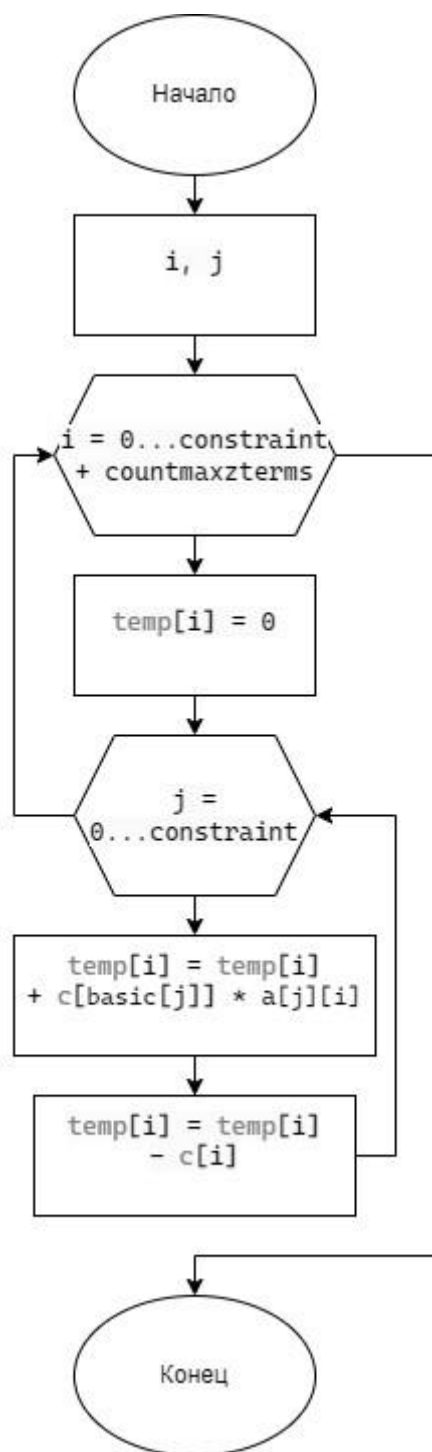
Ответ:

$$x_1 = 0, x_2 = 39/4, x_3 = 0, x_4 = 33/4$$

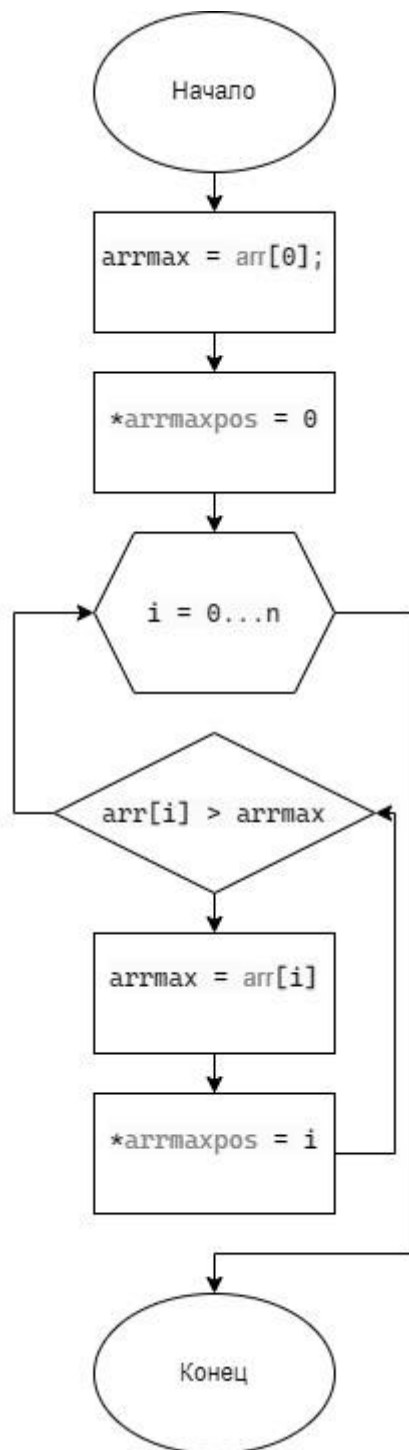
$$F(X) = -405/4$$

Блок-схемы:

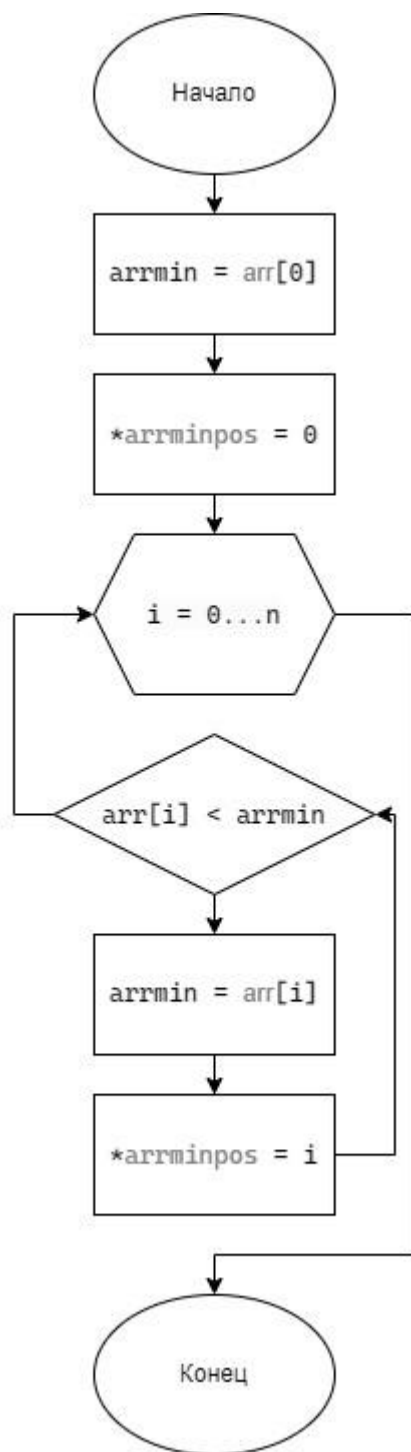
//Функция для поиска подсчёта



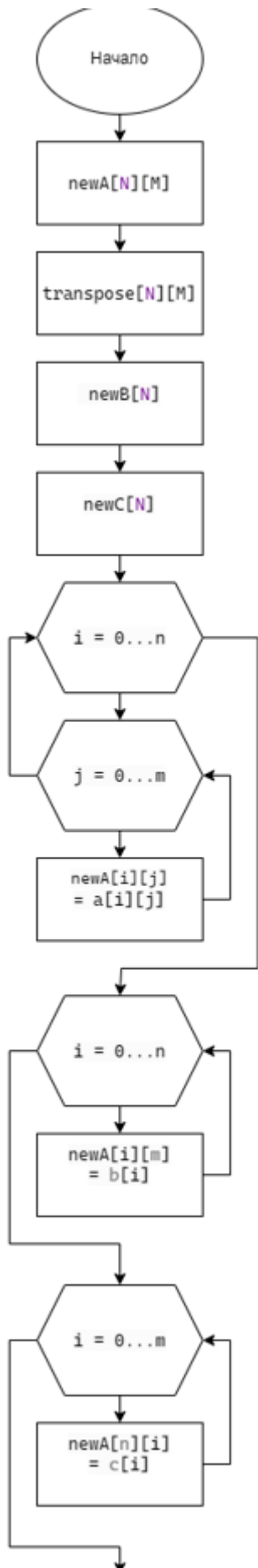
//Функция для поиска максимума

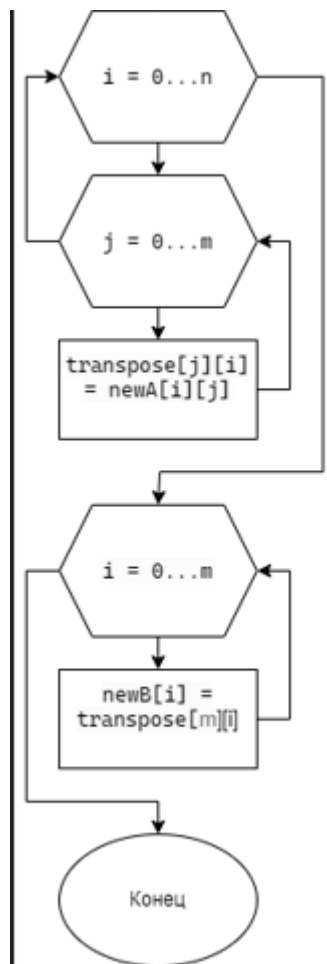


//Функция для поиска минимума



//Функция для поиска двойственного решения





Код программы:

```
#include <stdio.h>
#include <conio.h>
#include <Windows.h>
#include <locale.h>

#define INFINITY -999
#define N 10
#define M 20

void minimum(float* arr, int* arrminpos, int n);
/* Вычисляет позицию с минимальным значением среди массива arr, состоящего из n
элементов. */

void maximum(float* arr, int* arrminpos, int n);
/* Вычисляет позицию с минимальным значением среди массива arr, состоящего из n
элементов. */

void display(float c[], float b[], float a[][M], int basic[]);
/* Отображение таблицы */

void displayframe(float c[M]);
/* Отображает рамку таблицы */

void calctemp(float*, float[][M], float[], int[]);
/* Вычисляет Z */

int countmaxzterms;
int constraint;

// Функция для составления двойственной задачи
void outputDualTask(float a[N][M], float b[], float c[], int n, int m)
{
    float newA[N][M];
    float transpose[N][M];
    float newB[N];
    float newC[N];

    for (size_t i = 0; i < n; i++)
    {
        for (size_t j = 0; j < m; j++)
        {
            newA[i][j] = a[i][j];
        }
    }

    for (size_t i = 0; i < n; i++)
        newA[i][m] = b[i];

    for (size_t i = 0; i < m; i++)
        newA[n][i] = c[i];

    for (int i = 0; i <= n; ++i)
        for (int j = 0; j <= m; ++j) {
            transpose[j][i] = newA[i][j];
        }

    for (int i = 0; i < m; ++i) {
        newB[i] = transpose[m][i];
    }

    for (int i = 0; i < m; ++i) {
        newC[i] = transpose[i][n];
    }
}
```

```

printf("\n\n=== Составление двойственной задачи ===\n");

printf("\nПолученная функция:\n");
printf("\nf_min = ");

for (int i = 0; i < n; i++)
{
    if (i == 0)
        printf("%g*y%d", newB[i], i + 1);
    else
        printf(" + %g*y%d", newB[i], i + 1);
}

for (int i = 0; i < m; i++)
{
    for (int j = 0; j < n; j++)
    {
        if (j == 0)
            printf(" \n%g*y%d ", transpose[i][j], j + 1);
        else
            printf(" + %g*y%d ", transpose[i][j], j + 1);
    }

    printf(" <= %g\n", newC[i]);
}
}

void main()
{
    setlocale(0, "RUS");

    float c[M];          /* Сохраняет коэффициент целевой функции Max(z) */

    float a[N][M];       /* Сохраняет коэффициент ограничений */
    float b[N];          /* Сохраняет значения в правой части ограничений */
    float temp[M];       /* Сохраняет значения Zj-Cj */
    int bminpos;         /* Сохраняет минимальное значение позиции {Zj-Cj}, т.е.
входящей в переменную */
    float maxratio[M];   /* Сохраняет значение отношения Zj-Cj/a[i][j] */
    int maxratio_maxpos; /* Сохраняет минимальное значение позиции b[i]/a[i][j],
т.е. исходящей переменной */
    float key;           /* Сохраняет ключевой элемент */
    int gooutcol;        /* Сохраняет номер столбца, который выходит */
    int incomingcol;
    float z;             /* Сохраняет значение целевой функции */
    float x[M];
    int i, j;
    int basic[N];        /* Сохраняет базовую переменную */
    int flag = 0;

    /*** Инициализация базовых переменных ***/
    for (i = 0; i < M; i++)
        c[i] = x[i] = temp[i] = 0;

    for (i = 0; i < N; i++)
        for (j = 0; j < M; j++)
            a[i][j] = 0;

    /*** Ввод необходимых данных ***/
    printf("\nВведите задачу в стандартной форме\n");
    printf("\nВведите количество переменных целевой функции\n");
    scanf_s("%d", &countmaxzterms);
    printf("\nВведите коэффициенты целевой функции\n");

    for (i = 0; i < countmaxzterms; i++)

```

```

scanf_s("%f", &c[i]);

printf("\nВведенная функция:\n");
printf("\Max z = ");

for (i = 0; i < countmaxzterms; i++)
{
    if (i == 0)
        printf("%g*x%d", c[i], i + 1);
    else
        printf(" + %g*x%d", c[i], i + 1);
}

printf("\nВведите количество ограничений: \n");
scanf_s("%d", &constraint);

printf("\nВведите коэффициенты ограничений: \n");

for (i = 0; i < constraint; i++)
    for (j = 0; j < countmaxzterms; j++)
        scanf_s("%f", &a[i][j]);

for (i = 0; i < constraint; i++)
    a[i][j++] = 1;

printf("\nВведите значения свободных членов: \n");

for (i = 0; i < constraint; i++)
    scanf_s("%f", &b[i]);

for (i = 0; i < countmaxzterms + constraint; i++)
    basic[i] = (i + countmaxzterms);

printf("\nВы ввели: \n");

for (i = 0; i < constraint; i++)
{
    for (j = 0; j < countmaxzterms; j++)
    {
        if (j == 0)
            printf(" %g*x%d ", a[i][j], j + 1);
        else
            printf(" + %g*x%d ", a[i][j], j + 1);
    }
    printf(" <= %g\n", b[i]);
}

outputDualTask(a, b, c, constraint, countmaxzterms);

_getch();

/** Расчет для фактической таблицы */
do
{
    /** Завершающее условие */
    for (i = 0; i < constraint; i++)
    {
        flag = 1;

        if (b[i] <= 0)
        {
            flag = 0;

            break;

```

```

    }
}

z = 0;
calctemp(temp, a, c, basic);
printf("\n");

display(c, b, a, basic);
printf("\n\tZj-Cj\t\t");

for (i = 0; i < constraint + countmaxzterms; i++)
    printf("%0.3g\t", temp[i]);

printf("\n\n");

/**/ Определение исходящего столбца ***/
maximum(b, &bminpos, constraint);
gooutcol = basic[bminpos];

/**/ Определение входящего столбца ***/
for (i = 0; i < M; i++)
    maxratio[i] = INFINITY;

for (i = 0; i < constraint + countmaxzterms; i++)
{
    if (a[bminpos][i] == 0)
    {
        maxratio[i] = INFINITY;
        continue;
    }

    if (a[bminpos][i] > 0)
    {
        maxratio[i] = INFINITY;
        continue;
    }

    maxratio[i] = temp[i] / a[bminpos][i];
}

minimum(maxratio, &maxratiomaxpos, 2 * constraint);
incomingcol = maxratiomaxpos;

for (i = 0; i < constraint + countmaxzterms; i++)
    x[i] = 0;

for (i = 0; i < constraint; i++)
{
    x[basic[i]] = b[i];
    printf("y[%d]=%0.3g\n", basic[i] + 1, c[i]);
}

printf("\nРешений нет! Все отношения < 0!");

for (i = 0; i < constraint; i++)
    z = z + c[i] * x[i];

printf("Min(z) = %g", z);
printf("\nВходит в базис переменная: X%d\t", incomingcol + 1);
printf("Выходит из базиса переменная: X%d\n", gooutcol + 1);

basic[bminpos] = incomingcol;

/**/ Выполнение операций по приведению похожих выражений в
входящая переменная как исходящая переменная строковыми операциями ***/

```



```

        key = a[bminpos][incomingcol];
        b[bminpos] = b[bminpos] / key;

        for (i = 0; i < constraint + countmaxzterms; i++)
            a[bminpos][i] = a[bminpos][i] / key;

        for (i = 0; i < constraint; i++)
        {
            if (bminpos == i)
                continue;

            key = a[i][incomingcol];

            for (j = 0; j < (constraint + countmaxzterms); j++)
                a[i][j] = a[i][j] - a[bminpos][j] * key;
            b[i] = b[i] - b[bminpos] * key;
        }
        _getch();
    } while (flag == 0);

    printf("\nPress any key to exit...\n");
    _getch();
}

void calctemp(float* temp, float a[N][M], float c[M], int basic[N])
{
    int i, j;

    for (i = 0; i < constraint + countmaxzterms; i++)
    {
        temp[i] = 0;

        for (j = 0; j < constraint; j++)
            temp[i] = temp[i] + c[basic[j]] * a[j][i];

        temp[i] = temp[i] - c[i];
    }
}

void maximum(float* arr, int* arrmaxpos, int n)
{
    int i;
    int arrmax;
    arrmax = arr[0];
    *arrmaxpos = 0;

    for (i = 0; i < n; i++)
        if (arr[i] > arrmax)
        {
            arrmax = arr[i];
            *arrmaxpos = i;
        }
}

void minimum(float* arr, int* arrminpos, int n)
{
    int i;
    int arrmin;
    arrmin = arr[0];
    *arrminpos = 0;

    for (i = 0; i < n; i++)
        if (arr[i] < arrmin)
        {

```

```

        arrmin = arr[i];
        *arrminpos = i;
    }
}

void display(float c[N], float b[N], float a[N][M], int basic[N])
{
    int i, j;
    displayframe(c);

    for (i = 0; i < constraint; i++)
    {
        printf("\n%0.3g\tX%d\t%0.3g\t", c[basic[i]], basic[i] + 1, b[i]);

        for (j = 0; j < constraint + countmaxzterms; j++)
            printf("%0.3g\t", a[i][j]);

        printf("\n");
    }
}

void displayframe(float c[M])
{
    int i;
    printf("\t\tc[j]\t");

    for (i = 0; i < constraint + countmaxzterms; i++)
        printf("%0.2g\t", c[i]);

    printf("\n");
    printf("\nc[b]\tB\tb\t");

    for (i = 0; i < constraint + countmaxzterms; i++)
        printf("y[%d]\t", i + 1);

    printf("\n");
}

```

Введите задачу в стандартной форме

Введите количество переменных целевой функции

4

Введите коэффициенты целевой функции

2 -7 -1 -4

Введенная функция:

Max $z = 2x_1 + -7x_2 + -1x_3 + -4x_4$

Введите количество ограничений:

3

Введите коэффициенты ограничений:

-3 1 -4 1 -2 -3 -4 -2 4 5 2 -3

Введите значения свободных членов:

18 -32 24

Вы ввели:

$-3x_1 + 1x_2 + -4x_3 + 1x_4 \leq 18$

$-2x_1 + -3x_2 + -4x_3 + -2x_4 \leq -32$

$4x_1 + 5x_2 + 2x_3 + -3x_4 \leq 24$

=== Составление двойственной задачи ===

Полученная функция:

$f_{\max} = 18y_1 + -32y_2 + 24y_3$

$-3y_1 + -2y_2 + 4y_3 \leq 2$

$1y_1 + -3y_2 + 5y_3 \leq -7$

$-4y_1 + -4y_2 + 2y_3 \leq -1$

$1y_1 + -2y_2 + -3y_3 \leq -4$

		$c[j]$	2	-7	-1	-4	0	0	0
$c[b]$	B	b	$y[1]$	$y[2]$	$y[3]$	$y[4]$	$y[5]$	$y[6]$	$y[7]$
0	X5	18	-3	1	-4	1	1	0	0
0	X6	-32	-2	-3	-4	-2	0	1	0
0	X7	24	4	5	2	-3	0	0	1
	Zj-Cj		-2	7	1	4	0	0	0

$y[5]=2$

$y[6]=-7$

$y[7]=-1$

Конечный результат:

```
Z_max = -101,25  
x1 = 0, x2 = 9,75, x3 = 0, x4 = 8,25  
F_Min = -1,33  
y1 = 0  
y2 = 0,66  
y3 = 0,83
```

Вывод

В результате исследования стало ясно, что теория двойственности играет ключевую роль в оптимизационных задачах, позволяя перенести информацию о решении прямой задачи на ее двойственную форму и наоборот. Двойственный симплекс-метод является мощным инструментом для решения пары симметрично двойственных задач, позволяя эффективно итеративно находить оптимальное решение. Однако следует учитывать, что его эффективность может зависеть от структуры и размера задачи.

Метод последовательного уточнения оценок также представляет собой важный подход к решению линейных программ, особенно в случае задач с ограниченными ресурсами и необходимостью точного определения оптимального решения. Его применение может быть особенно полезным в случаях, когда другие методы оказываются менее эффективными или требуют слишком больших вычислительных ресурсов.

Таким образом, результаты данного исследования подчеркивают важность понимания и применения теории двойственности и связанных с ней методов оптимизации в контексте линейного программирования. Дальнейшие исследования в этой области могут быть направлены на улучшение эффективности методов решения, адаптацию их к конкретным классам задач и разработку новых подходов на основе существующих концепций.